Tailor Order Management System — Backend Development Plan

Stack: Node.js + Express + PostgreSQL (Sequelize ORM)
Deployment: Render
Goal: Replace all 80+ MSW mock handlers with a production backend

---

System Overview (What We're Building)

The backend supports a complex textile manufacturing workflow with:

- **11 user roles** (Admin, Sales, Fabrication, Production Head, Packet Creator, Dyeing, Worker, QA, Purchaser, Dispatch, Custom)
- **50+ granular permissions** across 12 modules
- **25+ order item statuses** with section-level independent tracking
- **6 inventory categories** (FABRIC, RAW_MATERIAL, MULTI_HEAD, ADDA_MATERIAL, READY_STOCK, READY_SAMPLE)
- **Shopify integration** with webhook sync
- **Notification system** for role-based alerts
- **Dual fulfillment paths** (Ready Stock vs Production)

Phase 1: Project Setup & Infrastructure

Goal: Backend project scaffold, tooling, environment config, and database connection

1A: Project Structure

```
server/
├── src/
│   ├── config/
│   │   ├── database.js        # Sequelize connection
│   │   ├── environment.js     # ENV validation (dotenv)
│   │   └── cors.js            # CORS config
│   ├── models/                # Sequelize models (1 file per table)
│   ├── migrations/            # Sequelize CLI migrations
│   ├── seeders/               # Seed data (mirrors mockData)
```

```
│   ├── routes/              # Express routers per module
│   ├── controllers/         # Request handlers
│   ├── services/            # Business logic layer
│   ├── middleware/
│   │   ├── auth.js          # JWT verification
│   │   ├── permissions.js   # Permission checker
│   │   ├── errorHandler.js  # Global error handler
│   │   ├── validate.js      # Request validation (Joi/Zod)
│   │   └── rateLimiter.js   # Rate limiting
│   ├── utils/
│   │   ├── statusMachine.js # Status transition validator
│   │   ├── bomCalculator.js # BOM material requirement calc
│   │   ├── notifications.js # Notification creator helper
│   │   └── pagination.js    # Standard pagination helper
│   ├── jobs/                # Background jobs (optional - Bull/Agenda)
│   └── app.js               # Express app setup
├── .env.example
├── .sequelizerc             # Sequelize CLI paths
├── package.json
└── server.js                # Entry point
```

1B: Dependencies

```
Core:       express, cors, helmet, morgan, dotenv, compression
Database:   sequelize, pg, pg-hstore
Auth:       jsonwebtoken, bcryptjs, cookie-parser
Validation: joi (or zod)
File Upload: multer, cloudinary (for QA videos/images)
Shopify:    @shopify/shopify-api
Jobs:       node-cron (or bull for Redis-backed queues)
Notifications: socket.io (real-time), nodemailer (email)
Dev:        nodemon, sequelize-cli, eslint, prettier
```

1C: Environment Configuration

```env
```

```
# Database
DATABASE_URL=postgresql://user:pass@host:5432/tailor_mgmt
DB_SSL=true

# Auth
JWT_ACCESS_SECRET=xxx
JWT_REFRESH_SECRET=xxx
ACCESS_TOKEN_EXPIRY=15m
REFRESH_TOKEN_EXPIRY=7d

# Shopify
SHOPIFY_STORE_URL=xxx
SHOPIFY_API_KEY=xxx
SHOPIFY_API_SECRET=xxx
SHOPIFY_WEBHOOK_SECRET=xxx

# File Upload
CLOUDINARY_URL=xxx

# Notifications
SMTP_HOST=xxx
SMTP_USER=xxx
SMTP_PASS=xxx

# App
PORT=5000
NODE_ENV=development
FRONTEND_URL=http://localhost:5173
```

Deliverable

Backend boots, connects to PostgreSQL, health-check endpoint works, CORS configured for frontend.

Phase 2: Database Schema & Migrations

Goal: Design and create all 20+ database tables with proper relationships

2A: Core Tables

users

Column	Type	Notes
id	UUID (PK)	
name	VARCHAR(255)	
email	VARCHAR(255) UNIQUE	
password_hash	VARCHAR(255)	bcrypt
role	ENUM	ADMIN, SALES, FABRICATION, PRODUCTION_HEAD, PACKET_CREATOR, DYEING, WORKER, QA, PURCHASER, DISPATCH, CUSTOM
phone	VARCHAR(50)	
permissions	JSONB	Array of permission strings
is_active	BOOLEAN	Default true
refresh_token_hash	VARCHAR(255)	For token rotation
last_login_at	TIMESTAMP	
created_at / updated_at	TIMESTAMP	

products

Column	Type	Notes
id	UUID (PK)	
name	VARCHAR(255)	
sku	VARCHAR(100) UNIQUE	
description	TEXT	
category	VARCHAR(100)	
shopify_product_id	VARCHAR(100)	Nullable
images	JSONB	Array of image URLs
is_active	BOOLEAN	
created_at / updated_at	TIMESTAMP	

boms (Bill of Materials - versioned per product)

Column	Type	Notes
id	UUID (PK)	
product_id	UUID (FK → products)	
version	INTEGER	Auto-increment per product
is_active	BOOLEAN	Only one active per product
size	VARCHAR(50)	Nullable (default or size-specific)
created_by	UUID (FK → users)	
created_at / updated_at	TIMESTAMP	

bom_items

Column	Type	Notes
id	UUID (PK)	
bom_id	UUID (FK → boms)	
inventory_item_id	UUID (FK → inventory_items)	
piece	VARCHAR(100)	e.g., "shirt", "dupatta", "sharara"
quantity_per_unit	DECIMAL(10,4)	Amount per 1 product unit
unit	VARCHAR(50)	meters, pieces, etc.
notes	TEXT	

2B: Inventory Tables

inventory_items

Column	Type	Notes
id	UUID (PK)	
name	VARCHAR(255)	
sku	VARCHAR(100) UNIQUE	
category	ENUM	FABRIC, RAW_MATERIAL, MULTI_HEAD, ADDA_MATERIAL, READY_STOCK, READY_SAMPLE
unit	VARCHAR(50)	meters, pieces, kg, etc.
remaining_stock	DECIMAL(12,4)	Current available
min_stock_level	DECIMAL(12,4)	Low stock threshold
unit_price	DECIMAL(12,2)	
vendor_name	VARCHAR(255)	
vendor_contact	VARCHAR(255)	
rack_location	VARCHAR(100)	Physical location
image_url	TEXT	
linked_product_id	UUID (FK → products)	Only for READY_STOCK items
is_active	BOOLEAN	
created_at / updated_at	TIMESTAMP	

inventory_movements

Column	Type	Notes
id	UUID (PK)	
inventory_item_id	UUID (FK)	
movement_type	ENUM	STOCK_IN, STOCK_OUT, RESERVED, ADJUSTMENT
quantity	DECIMAL(12,4)	Always positive
remaining_after	DECIMAL(12,4)	Stock after movement
reference_type	VARCHAR(100)	ORDER, PURCHASE, ADJUSTMENT, etc.
reference_id	UUID	Order ID, PO ID, etc.
notes	TEXT	
performed_by	UUID (FK → users)	
transaction_date	TIMESTAMP	
created_at	TIMESTAMP	

2C: Order Tables

orders

Column	Type	Notes
id	UUID (PK)	
order_number	VARCHAR(50) UNIQUE	Auto-generated (e.g., ORD-2025-0001)
status	VARCHAR(50)	ORDER_STATUS enum values
customer_name	VARCHAR(255)	
customer_email	VARCHAR(255)	
customer_phone	VARCHAR(50)	
shipping_address	JSONB	{ line1, line2, city, state, country, postal_code }
source	ENUM	MANUAL, SHOPIFY
shopify_order_id	VARCHAR(100)	Nullable
shopify_order_number	VARCHAR(50)	
shopify_sync_status	VARCHAR(50)	SYNCED, PENDING, FAILED
shopify_last_synced_at	TIMESTAMP	
fulfillment_source	ENUM	READY_STOCK, PRODUCTION, null
sales_owner_id	UUID (FK → users)	
production_head_id	UUID (FK → users)	Nullable
subtotal	DECIMAL(12,2)	
discount	DECIMAL(12,2)	
shipping_cost	DECIMAL(12,2)	
tax	DECIMAL(12,2)	
total	DECIMAL(12,2)	
payment_status	VARCHAR(50)	PENDING, PARTIAL, PAID
payment_method	VARCHAR(50)	

Column	Type	Notes
internal_notes	TEXT	
expected_delivery_date	DATE	
actual_delivery_date	DATE	
dispatched_at	TIMESTAMP	
dispatch_courier	VARCHAR(255)	
dispatch_tracking	VARCHAR(255)	
feedback_rating	INTEGER	1-5
feedback_text	TEXT	
created_at / updated_at	TIMESTAMP	

order_items

Column	Type	Notes
id	UUID (PK)	
order_id	UUID (FK → orders)	
product_id	UUID (FK → products)	
product_name	VARCHAR(255)	Denormalized
product_sku	VARCHAR(100)	Denormalized
quantity	INTEGER	
unit_price	DECIMAL(12,2)	
size_type	ENUM	STANDARD, CUSTOM
size	VARCHAR(50)	S, M, L, XL, etc.
height_range	VARCHAR(50)	e.g., "5'0-5'2"
status	VARCHAR(50)	ORDER_ITEM_STATUS values
fulfillment_source	ENUM	READY_STOCK, PRODUCTION
bom_id	UUID (FK → boms)	The BOM version used
modesty	BOOLEAN	
notes	TEXT	
section_statuses	JSONB	Per-section status tracking
created_at / updated_at	TIMESTAMP	

order_item_sections

Column	Type	Notes
id	UUID (PK)	
order_item_id	UUID (FK → order_items)	
piece	VARCHAR(100)	"shirt", "farshi", "dupatta", etc.
type	ENUM	MAIN, ADD_ON
status	VARCHAR(50)	SECTION_STATUS values
status_updated_at	TIMESTAMP	
status_updated_by	UUID (FK → users)	

order_item_custom_boms **(for CUSTOM size orders)**

Column	Type	Notes
id	UUID (PK)	
order_item_id	UUID (FK)	
inventory_item_id	UUID (FK)	
piece	VARCHAR(100)	
quantity	DECIMAL(10,4)	
unit	VARCHAR(50)	
created_by	UUID (FK → users)	Fabrication user
created_at / updated_at	TIMESTAMP	

2D: Customer Form Tables

customer_forms

Column	Type	Notes
id	UUID (PK)	
order_item_id	UUID (FK)	
form_type	ENUM	STANDARD, CUSTOM
version	INTEGER	Increments on revisions
status	ENUM	DRAFT, SENT, APPROVED, REJECTED
measurements	JSONB	All measurements data
custom_details	JSONB	Custom form extra fields
generated_by	UUID (FK → users)	
approved_by	UUID (FK → users)	
approved_at	TIMESTAMP	
rejection_reason	TEXT	
created_at / updated_at	TIMESTAMP	

measurement_charts

Column	Type	Notes
id	UUID (PK)	
chart_type	ENUM	SIZE, HEIGHT
data	JSONB	The chart matrix data
updated_by	UUID (FK → users)	
created_at / updated_at	TIMESTAMP	

2E: Packet Tables

packets

Column	Type	Notes
id	UUID (PK)	
order_item_id	UUID (FK)	
status	ENUM	PENDING, ASSIGNED, IN_PROGRESS, COMPLETED, VERIFIED, REJECTED
assigned_to	UUID (FK → users)	Packet creator
checked_by	UUID (FK → users)	Production head
started_at	TIMESTAMP	
completed_at	TIMESTAMP	
verified_at	TIMESTAMP	
rejection_reason	TEXT	
notes	TEXT	
created_at / updated_at	TIMESTAMP	

packet_items (pick list)

Column	Type	Notes
id	UUID (PK)	
packet_id	UUID (FK)	
inventory_item_id	UUID (FK)	
section_piece	VARCHAR(100)	
required_qty	DECIMAL(10,4)	
picked_qty	DECIMAL(10,4)	
unit	VARCHAR(50)	
is_picked	BOOLEAN	
picked_at	TIMESTAMP	
picked_by	UUID (FK → users)	

2F: Dyeing Tables

dyeing_tasks

Column	Type	Notes
id	UUID (PK)	
order_item_id	UUID (FK)	
assigned_to	UUID (FK → users)	Dyeing user
status	ENUM	PENDING, ACCEPTED, IN_PROGRESS, COMPLETED, REJECTED
sections	JSONB	Array of section pieces and their dyeing status
accepted_at	TIMESTAMP	
started_at	TIMESTAMP	
completed_at	TIMESTAMP	
rejection_reason	TEXT	
notes	TEXT	
created_at / updated_at	TIMESTAMP	

2G: Production Tables

production_tasks

Column	Type	Notes
id	UUID (PK)	
order_item_id	UUID (FK)	
section_piece	VARCHAR(100)	Which section this task is for
task_type	VARCHAR(100)	ADDA_WORK, CUTTING_WORK, EMBROIDERY_WORK, etc.
custom_task_name	VARCHAR(255)	For CUSTOM type
sequence_order	INTEGER	Order within section
assigned_to	UUID (FK → users)	Worker
status	ENUM	PENDING, READY, IN_PROGRESS, COMPLETED
planned_start	TIMESTAMP	
planned_end	TIMESTAMP	
actual_start	TIMESTAMP	
actual_end	TIMESTAMP	
notes	TEXT	
round	INTEGER	Default 1, increments for rework
created_by	UUID (FK → users)	Production head
created_at / updated_at	TIMESTAMP	

2H: QA & Approval Tables

qa_reviews

Column	Type	Notes
id	UUID (PK)	
order_item_id	UUID (FK)	
section_piece	VARCHAR(100)	
status	ENUM	PENDING, APPROVED, REJECTED
video_url	TEXT	YouTube URL
photos	JSONB	Array of photo URLs
reviewer_id	UUID (FK → users)	QA user
reviewed_at	TIMESTAMP	
rejection_reason	TEXT	
round	INTEGER	Tracks rework rounds
created_at / updated_at	TIMESTAMP	

client_approvals

Column	Type	Notes
id	UUID (PK)	
order_item_id	UUID (FK)	
status	ENUM	PENDING, APPROVED, REJECTED, ALTERATION_REQUESTED
video_url	TEXT	
screenshots	JSONB	Approval evidence
approved_by	UUID (FK → users)	Sales user who confirmed
rejection_reason	TEXT	
alteration_notes	TEXT	
created_at / updated_at	TIMESTAMP	

2I: Procurement Tables

procurement_demands

Column	Type	Notes
id	UUID (PK)	
order_item_id	UUID (FK)	
inventory_item_id	UUID (FK)	
section_piece	VARCHAR(100)	
required_qty	DECIMAL(10,4)	
available_qty	DECIMAL(10,4)	At time of check
shortage_qty	DECIMAL(10,4)	
status	ENUM	PENDING, PARTIALLY_FULFILLED, FULFILLED, CANCELLED
fulfilled_at	TIMESTAMP	
created_at / updated_at	TIMESTAMP	

2J: Activity & Notification Tables

`order_activities` (Timeline)

Column	Type	Notes
id	UUID (PK)	
order_id	UUID (FK)	
order_item_id	UUID (FK)	Nullable
action	VARCHAR(100)	STATUS_CHANGE, NOTE_ADDED, TASK_CREATED, etc.
description	TEXT	Human-readable
metadata	JSONB	Extra context (old_status, new_status, etc.)
performed_by	UUID (FK → users)	
created_at	TIMESTAMP	

notifications

Column	Type	Notes
id	UUID (PK)	
user_id	UUID (FK → users)	Recipient
type	VARCHAR(100)	ORDER_STATUS_CHANGE, LOW_STOCK, TASK_ASSIGNED, etc.
title	VARCHAR(255)	
message	TEXT	
reference_type	VARCHAR(100)	ORDER, ORDER_ITEM, INVENTORY, TASK
reference_id	UUID	Links to the relevant entity
is_read	BOOLEAN	Default false
read_at	TIMESTAMP	
created_at	TIMESTAMP	

alerts

Column	Type	Notes
id	UUID (PK)	
type	ENUM	LOW_STOCK, MATERIAL_SHORTAGE, OVERDUE_ORDER, etc.
severity	ENUM	INFO, WARNING, CRITICAL
title	VARCHAR(255)	
message	TEXT	
reference_type	VARCHAR(100)	
reference_id	UUID	
target_role	VARCHAR(50)	Which role sees this
is_resolved	BOOLEAN	
resolved_by	UUID (FK → users)	
resolved_at	TIMESTAMP	
created_at	TIMESTAMP	

2K: Shopify Integration Table

shopify_sync_logs

Column	Type	Notes
id	UUID (PK)	
order_id	UUID (FK → orders)	
direction	ENUM	INCOMING, OUTGOING
event_type	VARCHAR(100)	ORDER_CREATED, STATUS_UPDATED, FULFILLMENT, etc.
payload	JSONB	Raw webhook/API data
status	ENUM	SUCCESS, FAILED, PENDING
error_message	TEXT	
created_at	TIMESTAMP	

Migration Order (respecting FK dependencies)

```
1. users
2. measurement_charts
3. products
4. boms → bom_items
5. inventory_items → inventory_movements
6. orders → order_items → order_item_sections
7. order_item_custom_boms
8. customer_forms
9. packets → packet_items
10. dyeing_tasks
11. production_tasks
12. qa_reviews
13. client_approvals
14. procurement_demands
15. order_activities
16. notifications
17. alerts
18. shopify_sync_logs
```

Deliverable

All tables created with proper indexes, foreign keys, and constraints. Seed data matching current mock

data loaded.

Phase 3: Authentication & Authorization

Goal: JWT auth with access + refresh tokens, role-based permission middleware

3A: Auth Endpoints (mirrors authHandlers.js)

Method	Endpoint	Description
POST	/api/auth/login	Email/password → access token + httpOnly refresh cookie
POST	/api/auth/refresh	Refresh cookie → new access token
POST	/api/auth/logout	Invalidate refresh token, clear cookie
GET	/api/auth/me	Get current user from token

3B: Implementation Details

- **Access Token:** JWT, 15-minute expiry, sent in response body, stored in localStorage on frontend
- **Refresh Token:** JWT, 7-day expiry, httpOnly Secure SameSite=Strict cookie
- **Password:** bcrypt hash with salt rounds of 12
- **Token Rotation:** Each refresh invalidates the old refresh token
- **Permission Middleware:** `requirePermission('orders.create')` — checks user's permissions array

3C: Middleware Chain

```
request → rateLimiter → authenticate (JWT verify) → requirePermission → controller
```

Deliverable

Login/logout/refresh working. Frontend can swap from MSW to real API with no changes.

Phase 4: User Management Module

Goal: Admin CRUD for users with role/permission assignment

Endpoints (mirrors userHandlers.js)

Method	Endpoint	Permission	Description
GET	/api/users	users.view	List all users (with filters)
GET	/api/users/:id	users.view	User detail
POST	/api/users	users.create	Create user
PUT	/api/users/:id	users.edit	Update user (role, permissions, active)
DELETE	/api/users/:id	users.delete	Deactivate user (soft delete)
GET	/api/users/roles/templates	users.view	Get role permission templates

Deliverable

Admin can create/edit/deactivate users. Role templates auto-populate permissions.

Phase 5: Inventory Module

Goal: Complete inventory management across 6 categories

Endpoints (mirrors inventoryHandlers.js)

Method	Endpoint	Permission	Description
GET	/api/inventory-items	inventory.view	List with filters (category, vendor, stock status, search)
GET	/api/inventory-items/:id	inventory.view	Detail with movement history
POST	/api/inventory-items	inventory.create	Create item (any of 6 categories)
PUT	/api/inventory-items/:id	inventory.edit	Update item
DELETE	/api/inventory-items/:id	inventory.delete	Soft delete
POST	/api/inventory-items/:id/stock-in	inventory.stock_in	Record stock received

Method	Endpoint	Permission	Description
GET	/api/inventory/low-stock	inventory.view	Items below min threshold
GET	/api/inventory-items/ready-stock	inventory.view	Ready stock filtered by product

Business Logic

- Stock-in creates `inventory_movements` record and updates `remaining_stock`
- Low stock detection triggers `alerts` creation for PURCHASER role
- Ready stock items must have `linked_product_id`
- All stock changes are auditable via movements table

Deliverable

Full inventory CRUD, stock-in with audit trail, low-stock alerts.

Phase 6: Products & BOM Module

Goal: Product catalog with versioned BOMs

Endpoints (mirrors productHandlers.js)

Method	Endpoint	Permission	Description
GET	/api/products	products.view	List products
GET	/api/products/:id	products.view	Product detail with active BOM
POST	/api/products	products.create	Create product
PUT	/api/products/:id	products.edit	Update product
GET	/api/products/:id/bom	products.view	Get active BOM with items
PUT	/api/products/:id/bom	products.manage_bom	Update BOM (creates new version)
POST	/api/products/:id/bom-items	products.manage_bom	Add BOM item
GET	/api/products/:id/ready-stock	products.view	Available ready stock count

Business Logic

- BOM items restricted to categories: FABRIC, RAW_MATERIAL, MULTI_HEAD, ADDA_MATERIAL
- BOM versioning: updating creates new version, deactivates old
- BOM calculator utility: `calculateMaterialRequirements(orderItems)` → consolidated material list

Deliverable

Product CRUD, BOM management with versioning, BOM calculator utility.

Phase 7: Measurement Charts Module

Goal: Admin-editable global size and height charts

Endpoints

Method	Endpoint	Permission	Description
GET	/api/measurement-charts/size	measurements.view	Get size chart
PUT	/api/measurement-charts/size	measurements.edit	Update size chart
GET	/api/measurement-charts/height	measurements.view	Get height chart
PUT	/api/measurement-charts/height	measurements.edit	Update height chart
GET	/api/measurement-charts/default	measurements.view	Get both charts

Deliverable

Editable measurement charts used by customer form generation.

Phase 8: Orders Module (Core)

Goal: Order CRUD with two distinct creation paths (Manual + Shopify), automatic ready stock detection, and dual fulfillment routing

8A: Order CRUD Endpoints

Method	Endpoint	Permission	Description
GET	/api/orders	orders.view	List orders (filters: status, date, source, fulfillment_source, sales_owner, search)
GET	/api/orders/:id	orders.view	Order detail with items, sections, timeline, Shopify sync panel
POST	/api/orders	orders.create	Create order (manual — see 8C below)
PUT	/api/orders/:id	orders.edit	Update order
DELETE	/api/orders/:id	orders.delete	Cancel order (with ready stock return if applicable)
GET	/api/orders/:id/timeline	orders.view	Activity log
POST	/api/orders/:id/notes	orders.edit	Add internal note
GET	/api/orders/:id/ready-stock-issues	orders.view	Ready stock items issued to this order
GET	/api/orders/:id/material-requirements	orders.view	BOM material requirements (production orders)
POST	/api/orders/:id/check-ready-stock	orders.edit	Manual ready stock recheck

8B: Two Order Creation Paths

Orders enter the system through exactly two doors. Both converge at RECEIVED status and then go through the same automatic ready stock check logic.

Path 1: Manual Order Creation (by Sales)

```
Sales fills form → POST /api/orders (source: MANUAL)
  → Backend creates internal order (RECEIVED)
  → Backend calls Shopify API to create corresponding Shopify order
  → Saves shopify_order_id on internal order
  → Auto runs ready stock check (see 8D)
  → Routes to Ready Stock path OR Production path
```

Manual order payload includes everything needed for Shopify sync:

- Customer info (name, email, phone) — maps to Shopify customer
- Shipping address (line1, line2, city, state, country, postal_code) — required for Shopify fulfillment
- Order items (product_id, quantity, unit_price) — products have shopify_product_id
- Financials (discount, shipping_cost, tax, payment_status, payment_method)
- Internal notes (don't sync to Shopify)
- Optional "Force Production" flag — overrides ready stock, forces production path even if RS available

Path 2: Shopify Order (via Webhook or Manual Import)

```
Shopify webhook fires → POST /api/webhooks/shopify/orders/create (HMAC verified, no auth)
  OR
Admin clicks Import → POST /api/shopify/orders/:shopifyOrderId/import

  → Backend maps Shopify data to internal schema
  → Creates matching products if they don't exist in internal catalog
  → Creates internal order (RECEIVED, source: SHOPIFY, shopify_order_id saved)
  → Creates order_items with BOM mapping
  → Auto runs ready stock check (see 8D)
  → Routes to Ready Stock path OR Production path
```

Shopify webhook mapping:

- Shopify customer → customer_name, customer_email, customer_phone
- Shopify shipping_address → shipping_address JSONB
- Shopify line_items → order_items (matched via shopify_product_id on products table)
- Shopify financial_status → payment_status
- Shopify note → internal_notes

8C: Manual Order Creation Service (detailed logic)

```
1. Validate all product IDs exist and are active
2. For each order item:
   a. Find product, get active BOM
   b. Determine sections from BOM pieces (shirt, dupatta, sharara, etc.)
```

```
   c. Initialize section_statuses for each section
3. Calculate total = Σ(item.quantity × item.unit_price) - discount + shipping + tax
4. Create order record (status: RECEIVED, source: MANUAL)
5. Create order_items records with section tracking
6. Create order_item_sections records
7. Call Shopify API → create Shopify order → save shopify_order_id
   - If Shopify call fails: still create internal order, set shopify_sync_status = FAILED
   - Shopify sync failure should NOT block internal order creation
8. Log creation in order_activities
9. Notify Admin + Production Head (ORDER_CREATED)
10. Run ready stock check (8D) — this determines the next state
```

8D: Automatic Ready Stock Check (runs for BOTH paths)

When any order enters RECEIVED status, the system automatically determines the fulfillment path:

```
For each order_item:
  1. Find READY_STOCK inventory items where linked_product_id = item.product_id
  2. Sum total available ready stock across all matching inventory items
  3. Compare against item.quantity

Evaluate:
  - RS_OK: ALL order items have sufficient ready stock → READY STOCK PATH
  - RS_NEEDS_PRODUCTION: ANY item lacks ready stock → PRODUCTION PATH
  - If "Force Production" flag is set → always PRODUCTION PATH

READY STOCK PATH (RS_OK):
  → Set fulfillment_source = READY_STOCK on order + all items
  → Deduct ready stock: reduce remaining_stock on READY_STOCK inventory items
  → Create inventory_movements (type: ISSUE_READY_STOCK_TO_ORDER)
  → Move status: RECEIVED → AWAITING_CLIENT_APPROVAL
  → SKIP: inventory check, packet, dyeing, production, QA (all bypassed)
  → Notify Sales (READY_STOCK_FULFILLED)

PRODUCTION PATH (RS_NEEDS_PRODUCTION):
  → Set fulfillment_source = PRODUCTION on order + all items
  → Move status: RECEIVED → INVENTORY_CHECK
  → Follows full workflow: inventory check → packet → dyeing → production → QA → approval
  → Notify Purchaser (INVENTORY_CHECK_NEEDED)
```

8E: Shopify Sync Panel (on every order detail)

Every order, regardless of source, has a Shopify section in the API response:

```json
{
  "shopify": {
    "source": "MANUAL" | "SHOPIFY",
    "shopify_order_id": "12345" | null,
    "shopify_order_number": "#1008" | null,
    "shopify_order_url": "https://store.myshopify.com/admin/orders/12345",
    "sync_status": "SYNCED" | "PENDING" | "FAILED" | "NOT_SYNCED",
    "last_synced_at": "2025-03-01T10:00:00Z" | null
  }
}
```

- Manual orders not yet synced show "Sync to Shopify" action
- Already synced orders show "Re-sync" action
- Shopify-sourced orders always show the Shopify admin link

8F: Order Cancellation with Ready Stock Return

If a ready stock order is cancelled after stock was issued:

```
1. Find inventory_movements for this order (type: ISSUE_READY_STOCK_TO_ORDER)
2. Create reverse movements (type: RETURN_READY_STOCK)
3. Restore remaining_stock on the READY_STOCK inventory items
4. Update order status → CANCELLED
5. If Shopify-linked: cancel/update in Shopify
6. Log in timeline
```

8G: Customer Form Endpoints

Method	Endpoint	Permission	Description
GET	/api/orders/:id/customer-form	orders.view	Get form for order item
POST	/api/orders/:id/customer-form/generate	orders.manage_customer_forms	Generate form (standard uses global measurement chart, custom collects full

Method	Endpoint	Permission	Description
			measurements)
POST	/api/orders/:id/customer-form/send	orders.manage_customer_forms	Send for approval → status AWAITING_CUSTOMER_FORM_APPRO
PUT	/api/orders/:id/customer-form/approve	orders.approve_customer_forms	Approve → next step INVENTORY_CHECK
PUT	/api/orders/:id/customer-form/reject	orders.approve_customer_forms	Reject → new version, stays in AWAITING_CUSTOMER_FORM_APPRO

8H: Fabrication (Bespoke) Endpoints

Method	Endpoint	Permission	Description
GET	/api/fabrication/queue	fabrication.view	Custom size items tagged [custom_bom_required]
POST	/api/order-items/:id/custom-bom	fabrication.create_bom	Create custom BOM for this specific order item
PUT	/api/order-items/:id/custom-bom	fabrication.edit_bom	Edit custom BOM

Custom BOM is required when size_type = CUSTOM because standard BOMs can't be assumed for non-standard measurements.

8I: Status Machine Service

```javascript
```

```
// Validates that a status transition is allowed
// Note: RECEIVED has TWO exits depending on ready stock check result
const VALID_TRANSITIONS = {
  // Ready Stock path: RECEIVED → AWAITING_CLIENT_APPROVAL (skips everything)
  // Production path: RECEIVED → INVENTORY_CHECK or AWAITING_CUSTOMER_FORM_APPROVAL or FA
  RECEIVED: ['AWAITING_CLIENT_APPROVAL', 'INVENTORY_CHECK', 'AWAITING_CUSTOMER_FORM_A

  AWAITING_CUSTOMER_FORM_APPROVAL: ['INVENTORY_CHECK', 'AWAITING_CUSTOMER_FORM_AP
  FABRICATION_BESPOKE: ['INVENTORY_CHECK'],
  INVENTORY_CHECK: ['AWAITING_MATERIAL', 'CREATE_PACKET', 'READY_FOR_PRODUCTION'],
  AWAITING_MATERIAL: ['INVENTORY_CHECK'],
  CREATE_PACKET: ['PACKET_CHECK', 'PARTIAL_CREATE_PACKET'],
  PARTIAL_CREATE_PACKET: ['CREATE_PACKET', 'PACKET_CHECK'],
  PACKET_CHECK: ['READY_FOR_DYEING', 'CREATE_PACKET', 'READY_FOR_PRODUCTION'],
  PARTIAL_PACKET_CHECK: ['PACKET_CHECK', 'READY_FOR_DYEING'],
  READY_FOR_DYEING: ['IN_DYEING', 'PARTIALLY_IN_DYEING'],
  PARTIALLY_IN_DYEING: ['IN_DYEING'],
  IN_DYEING: ['DYEING_COMPLETED'],
  DYEING_COMPLETED: ['READY_FOR_PRODUCTION'],
  READY_FOR_PRODUCTION: ['IN_PRODUCTION'],
  IN_PRODUCTION: ['PARTIAL_IN_PRODUCTION', 'PRODUCTION_COMPLETED'],
  PARTIAL_IN_PRODUCTION: ['IN_PRODUCTION', 'PRODUCTION_COMPLETED'],
  PRODUCTION_COMPLETED: ['QA_PENDING'],
  QA_PENDING: ['QA_APPROVED', 'QA_REJECTED'],
  QA_REJECTED: ['IN_PRODUCTION'],
  ALL_SECTIONS_QA_APPROVED: ['VIDEO_UPLOADED'],
  VIDEO_UPLOADED: ['READY_FOR_CLIENT_APPROVAL'],
  READY_FOR_CLIENT_APPROVAL: ['AWAITING_CLIENT_APPROVAL'],
  AWAITING_CLIENT_APPROVAL: ['CLIENT_APPROVED', 'REWORK_REQUIRED', 'ALTERATION_REQUIR
  ALTERATION_REQUIRED: ['IN_PRODUCTION'],
  CLIENT_APPROVED: ['READY_FOR_DISPATCH'],
  READY_FOR_DISPATCH: ['DISPATCHED'],
  DISPATCHED: ['COMPLETED'],
  REWORK_REQUIRED: ['INVENTORY_CHECK', 'READY_FOR_PRODUCTION'],
}
```

8J: Orders List Filters

The orders list supports filtering by:

- **status** (multi-select)

- **source** (MANUAL / SHOPIFY)
- **fulfillment_source** (READY_STOCK / PRODUCTION)
- **sales_owner** (consultant)
- **date range** (created_at)
- **search** (order number, customer name, email)
- **payment_status** (PENDING / PARTIAL / PAID)

The list response includes a `fulfillment_source` badge and conditionally shows `current_stage` only for production orders.

Deliverable

Full order lifecycle with dual creation paths (Manual + Shopify), automatic ready stock detection, dual fulfillment routing, Shopify sync panel, customer forms, and fabrication queue. Status machine enforces all valid transitions including the ready stock shortcut.

Phase 9: Inventory Check & Procurement Module

Goal: Automated BOM-based material checking and procurement demand generation

Endpoints

Method	Endpoint	Permission	Description
POST	/api/order-items/:id/inventory-check	orders.edit	Run inventory check for an order item
GET	/api/orders/material-shortages	procurement.view	Dashboard of all shortages
GET	/api/procurement/demands	procurement.view	List procurement demands
PUT	/api/procurement/demands/:id/fulfill	procurement.manage	Mark demand fulfilled
POST	/api/order-items/:id/recheck-inventory	orders.edit	Re-run after stock-in

Business Logic (mirrors ordersHandlers.js inventory check)

1. Get BOM for order item (standard or custom)
2. For each section in the order item:
    - Filter BOM items by section piece

    - Calculate required qty × order quantity
    - Check against inventory_items.remaining_stock
    - If sufficient → section status = INVENTORY_PASSED
    - If shortage → section status = AWAITING_MATERIAL, create procurement_demand
3. Determine overall order item status based on section results
4. Create inventory_movements (RESERVED) for passed sections
5. Create alerts for shortages
6. Notify PURCHASER role

Deliverable

Auto inventory check on order creation, shortage detection, procurement demand dashboard.

Phase 10: Packet Workflow Module

Goal: Material packet creation and verification

Endpoints (mirrors packetHandlers.js)

Method	Endpoint	Permission	Description
GET	/api/packets	production.view	List packets
GET	/api/packets/my-tasks	production.view	Packets assigned to current user
GET	/api/packets/check-queue	production.approve_packets	Packets awaiting verification
GET	/api/order-items/:id/packet	production.view	Packet for an order item
POST	/api/order-items/:id/packet/assign	production.manage	Assign packet creator
POST	/api/order-items/:id/packet/start	production.view	Start picking
POST	/api/order-items/:id/packet/pick-item	production.view	Mark item picked
POST	/api/order-items/:id/packet/complete	production.view	Mark picking complete
POST	/api/order-items/:id/packet/approve	production.approve_packets	Verify packet
POST	/api/order-items/:id/packet/reject	production.approve_packets	Reject (with reason)

Business Logic

- Auto-generate pick list from BOM items per section
- Track individual item picking with deduction from inventory
- Rejection sends packet back with reason; section status reverts
- Approval advances section to PACKET_VERIFIED → next workflow step
- Supports partial packets (some sections ready, others waiting)

Deliverable

Complete packet creation → picking → verification pipeline.

Phase 11: Dyeing Module

Goal: Dyeing department workflow for sections that need dyeing

Endpoints (mirrors dyeingHandlers.js)

Method	Endpoint	Permission	Description
GET	/api/dyeing/available-tasks	dyeing.view	Sections ready for dyeing
GET	/api/dyeing/my-tasks	dyeing.view	Current user's accepted tasks
GET	/api/dyeing/completed-tasks	dyeing.view	Completed tasks
GET	/api/dyeing/task/:orderItemId	dyeing.view	Task detail
GET	/api/dyeing/stats	dyeing.view	Dashboard statistics
POST	/api/dyeing/task/:orderItemId/accept	dyeing.accept	Accept sections
POST	/api/dyeing/task/:orderItemId/start	dyeing.start	Start dyeing
POST	/api/dyeing/task/:orderItemId/complete	dyeing.complete	Complete dyeing
POST	/api/dyeing/task/:orderItemId/reject	dyeing.accept	Reject sections

Business Logic

- Sections in READY_FOR_DYEING appear in available tasks
- Dyeing user accepts → DYEING_ACCEPTED
- Start → DYEING_IN_PROGRESS
- Complete → DYEING_COMPLETED → section advances to READY_FOR_PRODUCTION
- Reject → sends back with reason (DYEING_REJECTED)
- Updates order item status based on section aggregate

Deliverable

Full dyeing workflow with section-level tracking.

Phase 12: Production Module

Goal: Production planning, task management, and worker task execution

Endpoints (mirrors production-related MSW handlers)

Method	Endpoint	Permission	Description
POST	/api/order-items/:id/assign-production-head	production.assign_head	Admin assigns prod head
GET	/api/order-items/:id/tasks	production.view	Get all tasks for order item
POST	/api/order-items/:id/tasks	production.assign_tasks	Create tasks (production head)
PUT	/api/order-items/:id/start-production	production.manage	Start production
GET	/api/tasks/my-tasks	production.view	Worker's assigned tasks
PUT	/api/tasks/:id/start	production.start_task	Worker starts task
PUT	/api/tasks/:id/complete	production.complete_task	Worker completes task
POST	/api/order-items/:id/sections/:piece/send-to-qa	production.send_to_qa	Send section to QA
GET	/api/production/board	production.view	Kanban board data

Business Logic

- Production head creates tasks per section with assigned workers and sequence
- Start production → first task in sequence becomes READY
- Task completion → check if all tasks at same sequence are done → advance to next
- All tasks done → section PRODUCTION_COMPLETED
- All sections done → order item PRODUCTION_COMPLETED
- Rework creates new round of tasks (round > 1)
- Production board aggregates across all active orders

Deliverable

Production planning, worker task execution, automatic stage progression.

Phase 13: QA & Client Approval Module

Goal: QA review with video uploads, client approval flow, rework/alteration loops

Endpoints

Method	Endpoint	Permission	Description
GET	/api/qa/queue	qa.view	Sections ready for QA
POST	/api/qa/order-items/:id/sections/:piece/review	qa.approve	Submit QA review
POST	/api/qa/order-items/:id/upload-video	qa.upload_video	Upload YouTube video URL
POST	/api/qa/order-items/:id/send-to-sales	qa.send_to_sales	Send to sales for client approval
GET	/api/sales/approval-queue	sales.view_approval_queue	Items awaiting client response
POST	/api/sales/order-items/:id/send-to-client	sales.send_to_client	Mark as sent to client
POST	/api/sales/order-items/:id/mark-approved	sales.mark_client_approved	Client approved

Method	Endpoint	Permission	Description
POST	/api/sales/order-items/:id/request-rework	sales.request_alteration	Client wants changes
POST	/api/sales/order-items/:id/request-revideo	sales.request_revideo	Request new QA video
POST	/api/sales/order-items/:id/cancel	sales.cancel_order	Client cancelled
POST	/api/sales/order-items/:id/approve-payment	sales.approve_payments	Verify payment received

Business Logic

- QA reviews each section independently (approve/reject with photos)
- All sections approved → QA uploads YouTube video for order item
- Video uploaded → READY_FOR_CLIENT_APPROVAL
- Sales sends to client → AWAITING_CLIENT_APPROVAL
- Client approved → CLIENT_APPROVED → payment verification → READY_FOR_DISPATCH
- Rework → REWORK_REQUIRED → back to production planning
- Alteration → ALTERATION_REQUIRED → specific sections go back

Deliverable

Complete QA → Client Approval → Rework loop.

Phase 14: Dispatch & Completion Module

Goal: Dispatch orders and collect feedback

Endpoints (mirrors dispatchHandlers.js)

Method	Endpoint	Permission	Description
GET	/api/dispatch/queue	dispatch.view	Orders ready for dispatch
GET	/api/dispatch/dispatched	dispatch.view	Already dispatched orders
GET	/api/dispatch/completed	dispatch.view	Completed orders

Method	Endpoint	Permission	Description
GET	/api/dispatch/stats	dispatch.view	Dashboard stats
POST	/api/dispatch/order/:orderId/dispatch	dispatch.manage	Mark dispatched (courier, tracking)
POST	/api/dispatch/order/:orderId/complete	dispatch.manage	Mark completed
POST	/api/orders/:id/feedback	orders.edit	Record customer feedback

Business Logic

- CLIENT_APPROVED + payment verified → READY_FOR_DISPATCH
- Dispatch captures courier, tracking, date
- Creates Shopify fulfillment via API
- DISPATCHED → COMPLETED (after feedback or manual close)

Deliverable

Dispatch workflow with Shopify fulfillment sync.

Phase 15: Shopify Integration Module

Goal: Bi-directional sync between Shopify and internal system

Endpoints

Method	Endpoint	Permission	Description
GET	/api/shopify/settings	admin	Get integration settings
POST	/api/shopify/test-connection	admin	Test Shopify API connection
POST	/api/shopify/webhooks/register	admin	Register webhooks
GET	/api/shopify/orders	orders.view	List Shopify orders (proxy)
POST	/api/shopify/orders/:shopifyOrderId/import	orders.create	Import Shopify order
POST	/api/orders/:id/sync-to-shopify	orders.edit	Push internal order to Shopify

Method	Endpoint	Permission	Description
POST	/api/orders/:id/resync-shopify	orders.edit	Force re-sync

Webhook Endpoints (no auth — verified by HMAC)

Method	Endpoint	Description
POST	/api/webhooks/shopify/orders/create	New order from Shopify
POST	/api/webhooks/shopify/orders/updated	Order updated in Shopify
POST	/api/webhooks/shopify/orders/cancelled	Order cancelled in Shopify

Business Logic

- Webhook receives Shopify order → create internal order + items → auto inventory check
- Internal status changes → sync to Shopify (tags, notes, fulfillment)
- HMAC signature verification on all webhooks
- All sync operations logged in shopify_sync_logs
- Retry mechanism for failed syncs

Deliverable

Full bi-directional Shopify sync with webhook handling and sync logging.

Phase 16: Notification System

Goal: Real-time and stored notifications for all role-based events

16A: Notification Endpoints

Method	Endpoint	Description
GET	/api/notifications	Get current user's notifications
GET	/api/notifications/unread-count	Get unread count
PUT	/api/notifications/:id/read	Mark as read

Method	Endpoint	Description
PUT	/api/notifications/read-all	Mark all as read
GET	/api/alerts	Get alerts for current user's role
PUT	/api/alerts/:id/resolve	Resolve an alert

16B: Real-time with Socket.IO

```javascript
// Server emits to specific user or role room
io.to(`user:${userId}`).emit('notification', { ... })
io.to(`role:PURCHASER`).emit('alert', { ... })
```

16C: Notification Triggers (auto-created by services)

Event	Recipients	Type
New order created	Admin, Production Head	ORDER_CREATED
Order needs inventory check	Purchaser	INVENTORY_CHECK_NEEDED
Material shortage detected	Purchaser	MATERIAL_SHORTAGE
Packet assigned	Packet Creator	PACKET_ASSIGNED
Packet ready for check	Production Head	PACKET_READY_CHECK
Dyeing task available	Dyeing users	DYEING_TASK_AVAILABLE
Production task assigned	Worker	TASK_ASSIGNED
Section sent to QA	QA users	QA_REVIEW_NEEDED
QA video uploaded	Sales	CLIENT_APPROVAL_NEEDED
Client approved	Dispatch	READY_FOR_DISPATCH
Client requested rework	Production Head	REWORK_NEEDED
Order dispatched	Sales, Admin	ORDER_DISPATCHED

Event	Recipients	Type
Low stock alert	Purchaser	LOW_STOCK
Overdue order alert	Admin, Sales	ORDER_OVERDUE

16D: Email Notifications (Optional Phase)

- Critical alerts (material shortage, rework) can trigger email
- Use nodemailer with configurable templates
- Preference per user: email on/off per notification type

Deliverable

Stored notifications with unread counts, real-time via Socket.IO, role-targeted alerts.

Phase 17: Admin Dashboard & Reports APIs

Goal: Aggregated data endpoints for admin dashboard widgets

Endpoints

Method	Endpoint	Permission	Description
GET	/api/dashboard/order-funnel	reports.view	Orders count by status
GET	/api/dashboard/production-pipeline	reports.view	Orders by production stage
GET	/api/dashboard/inventory-alerts	reports.view	Low stock / shortage summary
GET	/api/dashboard/qa-metrics	reports.view	QA approval/rework rates
GET	/api/dashboard/sales-summary	reports.view	Revenue by salesperson
GET	/api/dashboard/recent-activity	reports.view	Latest order activities
GET	/api/reports/order-completion	reports.view	Completion rate stats
GET	/api/reports/production-efficiency	reports.view	Avg time per stage
GET	/api/reports/material-consumption	reports.view	Material usage analytics

Deliverable

Dashboard and reporting data endpoints ready for frontend widgets.

Phase 18: Frontend Integration & Deployment

Goal: Connect frontend to real backend, deploy both

18A: Frontend Changes

- Remove MSW registration in production builds (already planned)
- Update `appConfig.apiBaseUrl` to point to backend URL
- Verify httpClient token refresh works with real cookies
- Test all API flows end-to-end

18B: Deployment (Render)

- **Backend:** Node.js web service on Render
- **Database:** Render PostgreSQL (or external like Supabase/Neon)
- **Frontend:** Static site on Render (or Vercel/Netlify)
- **Environment:** Production env vars in Render dashboard
- **SSL:** Automatic via Render

18C: Production Checklist

- [ ] Database migrations run cleanly
- [ ] Seed data loaded (admin user at minimum)
- [ ] All 80+ endpoints tested
- [ ] JWT refresh flow verified
- [ ] Shopify webhooks registered and verified
- [ ] Socket.IO working for notifications
- [ ] Error logging set up (consider Sentry)
- [ ] Rate limiting configured
- [ ] CORS locked to frontend domain
- [ ] Database backups configured

Deliverable

Production-ready deployment with frontend connected to real backend.

Recommended Build Order Summary

Phase	Module	Priority	Est. Effort
1	Project Setup & Infrastructure	🔴 Critical	1-2 days
2	Database Schema & Migrations	🔴 Critical	3-4 days
3	Authentication & Authorization	🔴 Critical	2-3 days
4	User Management	🔴 Critical	1-2 days
5	Inventory Module	🟠 High	2-3 days
6	Products & BOM Module	🟠 High	2-3 days
7	Measurement Charts	🟡 Medium	1 day
8	Orders Module (Core)	🔴 Critical	4-5 days
9	Inventory Check & Procurement	🟠 High	2-3 days
10	Packet Workflow	🟠 High	2-3 days
11	Dyeing Module	🟠 High	2 days
12	Production Module	🔴 Critical	3-4 days
13	QA & Client Approval	🟠 High	3-4 days
14	Dispatch & Completion	🟡 Medium	1-2 days
15	Shopify Integration	🟠 High	3-4 days
16	Notification System	🟠 High	2-3 days
17	Dashboard & Reports APIs	🟡 Medium	2-3 days
18	Frontend Integration & Deploy	🔴 Critical	2-3 days

Phase	Module	Priority	Est. Effort
Total			**~38-52 days**

Key Architectural Decisions

1. **Section-level status tracking** in both DB columns AND JSONB — sections table for querying, JSONB for quick reads
2. **Status machine as a service** — all transitions validated server-side, no direct status updates
3. **Activity logging** on every state change — complete audit trail
4. **Notification service** is cross-cutting — called from every module's service layer
5. **BOM calculator** is a shared utility used by inventory check, packet creation, and procurement
6. **Sequelize transactions** for multi-table operations (e.g., inventory check + stock deduction + demand creation)
7. **Soft deletes** for users, products, inventory items — never lose data
8. **UUID primary keys** throughout — no sequential IDs exposed